

Guía FP Ejercicio

Suma de N números naturales

Propósito de la guía. Acompañarte paso a paso para desarrollar el ejercicio de suma de números naturales, primero en PSeInt y luego en Code::Blocks, cuidando la lógica, la sintaxis y la prueba de escritorio. La idea es que no solo funcione, sino que entiendas por qué funciona.

Resultado esperado

Leer un número entero n y calcular la suma de los primeros n números naturales.

Contexto del ejercicio

En este ejercicio el programa solicita un número entero al usuario.

Luego, utilizando una estructura repetitiva Mientras, suma todos los números desde 1 hasta n.

La lógica de acumulación utilizada es:

```
suma=suma+i
```

1. Seudocódigo para PSeInt

Antes de programar en C, conviene escribir la solución en PSeInt. Aquí se observa la lógica con palabras cercanas al lenguaje natural.

Proceso SumaNNaturales

Definir n, i, suma Como Entero

Escribir "Ingrese un numero: "

Leer n

suma <- 0

i <- 1

Mientras i <= n Hacer

 suma <- suma + i

 i <- i + 1

FinMientras

Escribir "La suma es: ", suma

FinProceso

Tip amigable: Imagina que: i va recorriendo cada número natural, suma guarda el acumulado, en cada repetición el programa agrega el valor actual de i.

2. Prueba de escritorio en PSeInt

La prueba de escritorio permite comprobar manualmente qué hace el algoritmo. Para que sea fácil de revisar, se muestra el comportamiento.

i	Suma
1	1
2	3
3	6
4	10

Salida esperada:

```
Ingrese un número: 4
La suma es: 10
```

3. Guía para pasar de PSeInt a Code::Blocks con sintaxis validada

Ahora sí: pasamos la lógica a lenguaje C. La clave es traducir cada instrucción de PSeInt a su equivalente en C, sin perder el orden de los ciclos.

PSeInt	C / Code::Blocks	Para recordar
Proceso ... FinProceso	int main() { ... return 0; }	Todo programa en C inicia en main.
Definir variables reales	int n, i, suma;	Los números enteros se declaran con int.
Escribir	Printf();	Printf muestra mensajes y resultados.
Leer	Scanf();	Scanf permite ingresar datos.
Mientras	while	While repite mientras la condición sea verdadera.
suma <- suma + i	suma = suma + i;	Se utiliza una variable acumuladora.
i <- i + 1	i++;	Incrementa el contador del ciclo.

Código validado en C para Code::Blocks

```
#include <stdio.h>

int main()
{
    int n, i, suma;

    printf("Ingrese un numero: ");
    scanf("%d", &n);

    suma = 0;
    i = 1;

    while(i <= n)
```

```

{
    suma = suma + i;

    i++;
}

printf("La suma es: %d", suma);

return 0;
}

```

Pasos sugeridos en Code::Blocks

1. Crear un nuevo archivo con extensión .c.
2. Copiar el código validado en C.
3. Compilar con Build o F9
4. Ejecutar con Run o con Build and Run.
5. Ingresar diferentes valores para n.
6. Comparar los resultados con la prueba de escritorio.

Errores comunes que debes evitar:

- Olvidar inicializar suma en 0.
- Olvidar incrementar i.
- Usar = en lugar de < = en la condición.
- Declarar variables enteras como float.
- Olvidar el símbolo & en scanf.

4. Rúbrica de evaluación sobre 20 puntos

Esta rúbrica permite valorar el desarrollo completo del ejercicio, desde la comprensión del problema hasta la ejecución en Code::Blocks.

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Comprende correctamente el proceso de acumulación.	Comprende parcialmente la lógica.	Confunde acumulación y contador.	4 pts
Seudocódigo en PSeInt	Usa correctamente mientras y variables acumuladoras.	Presenta pequeños errores de sintaxis.	El algoritmo está incompleto.	4 pts
Prueba de escritorio	Comprueba correctamente los resultados.	Presenta una prueba parcial.	No evidencia seguimiento lógico.	4 pts
Traducción a C en Code::Blocks	El código compila y ejecuta correctamente.	Tiene errores menores corregibles.	El código no compila o falla.	5 pts
Presentación y orden	Trabajo limpio y organizado.	Comprensible con pequeños detalles.	Trabajo desordenado.	3 pts

Total: 20 puntos. Para una buena calificación, lo más importante es que exista coherencia entre pseudocódigo, prueba de escritorio, código en C y salida final.

Lista rápida de verificación antes de entregar

- El programa solicita un número entero.
- La variable suma inicia en 0.
- El contador i inicia en 1.
- El ciclo mientras funciona correctamente.
- El acumulador suma correctamente los valores.
- El código compila sin errores.